

Question **1**

Correct

Marked out of 1.00

Flag question

You are transporting some boxes through a tunnel, where each box is a parallelepiped, and is characterized by its length, width and height.

The height of the tunnel **41** feet and the width can be assumed to be infinite. A box can be carried through the tunnel only if its height is strictly less than the tunnel's height. Find the volume of each box that can be successfully transported to the other end of the tunnel. Note: Boxes cannot be rotated.

Input Format

The first line contains a single integer n , denoting the number of boxes.

n lines follow with three integers on each separated by single spaces - $length_i$ $width_i$ and $height_i$, which are length, width and height in feet of the i -th box.

Constraints

$$1 \leq n \leq 100$$

$$1 \leq length_i, width_i, height_i \leq 100$$

Output Format

For every box from the input which has a height lesser than **41** feet, print its volume in a separate line.

Sample Input 0

```
4
5 5 5
1 2 40
10 5 41
7 2 42
```

Sample Output 0

```
125
80
```

Explanation 0

The first box is really low, only **5** feet tall, so it can pass through the tunnel and its volume is **5 x 5 x 5 = 125**.

The second box is sufficiently low, its volume is $1 \times 2 \times 4 = 80$.

The third box is exactly **41** feet tall, so it cannot pass. The same can be said about the fourth box.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     scanf("%d", &n); // Read the number of boxes
6
7     for (int i = 0; i < n; i++) {
8         int length, width, height;
9         scanf("%d %d %d", &length, &width, &height);
10
11         // Check if the height of the box is less than 41
12         if (height < 41) {
13             int volume = length * width * height;
14             printf("%d\n", volume); // Print the volume of the box
15         }
16     }
17
18     return 0;
19 }
```

	Input	Expected	Got	
✓	4	125	125	✓
	5 5 5	80	80	
	1 2 40			
	10 5 41			
	7 2 42			

Passed all tests! ✓

Question 2

Correct

Marked out of 1.00

Flag question

You are given n triangles, specifically, their sides a_i , b_i and c_i . Print them in the same style but sorted by their areas from the smallest one to the largest one. It is guaranteed that all the areas are different.

The best way to calculate a volume of the triangle with sides a , b and c is Heron's formula:

$$S = \sqrt{p * (p - a) * (p - b) * (p - c)}$$
 where $p = (a + b + c) / 2$.

Input Format

First line of each test file contains a single integer n . n lines follow with a_i , b_i and c_i on each separated by single spaces.

Constraints

$1 \leq n \leq 100$
 $1 \leq a_i, b_i, c_i \leq 70$
 $a_i + b_i > c_i, a_i + c_i > b_i$ and $b_i + c_i > a_i$

Output Format

Print exactly n lines. On each line print 3 integers separated by single spaces, which are a_i , b_i and c_i of the corresponding triangle.

Sample Input 0

3
7 24 25
5 12 13
3 4 5

Sample Output 0

3 4 5
5 12 13
7 24 25

Explanation 0

```

1 #include <stdio.h>
2 #include <math.h>
3 #include <stdlib.h>
4
5 // Structure to hold the sides of a triangle and its area
6 typedef struct {
7     int a, b, c;
8     double area;
9 } Triangle;
10
11 // Function to calculate the area of a triangle using Heron's formula
12 double calculate_area(int a, int b, int c) {
13     double p = (a + b + c) / 2.0;
14     return sqrt(p * (p - a) * (p - b) * (p - c));
15 }
16
17 // Comparison function for sorting triangles by area
18 int compare(const void *t1, const void *t2) {
19     Triangle *triangle1 = (Triangle *)t1;
20     Triangle *triangle2 = (Triangle *)t2;
21     if (triangle1->area < triangle2->area)
22         return -1;
23     else if (triangle1->area > triangle2->area)
24         return 1;
25     else
26         return 0;
27 }
28
29 int main() {
30     int n;
31     scanf("%d", &n);
32
33     Triangle triangles[n];
34
35     for (int i = 0; i < n; i++) {
36         scanf("%d %d %d", &triangles[i].a, &triangles[i].b, &triangles[i].c);
37         triangles[i].area = calculate_area(triangles[i].a, triangles[i].b, triangles[i].c);
38     }
39
40     // Sort the triangles by area
41     qsort(triangles, n, sizeof(Triangle), compare);
42
43     // Print the sorted triangles
44     for (int i = 0; i < n; i++) {
45         printf("%d %d %d\n", triangles[i].a, triangles[i].b, triangles[i].c);
46     }
47
48     return 0;
49 }

```

	Input	Expected	Got	
✓	3	3 4 5	3 4 5	✓
	7 24 25	5 12 13	5 12 13	
	5 12 13	7 24 25	7 24 25	
	3 4 5			

Passed all tests! ✓